Introduction aux bases de données avec SQL

Enseignant :	Louis RAYNAL	**Établissement :**	ICES
Contact :	l-raynal@ices.fr	**Année :**	2024-2025
Public :	L3 Maths et M2 Bio-Santé		
Ressources :	https://github.com/LouisRaynal/coursSQL		

TP noté 1 (3h)

Nom :
Prénom :
Filière :

Rappels : Toute forme de communication entre vous est interdite. Toute triche résultera en la note de 0.

Documents autorisés : Polycopiés et scripts SQL des cours-TP, bases de données utilisées jusqu'à présent, vos notes de cours, fiches mémo., et autres ressources disponibles à l'adresse https://github.com/LouisRaynal/coursSQL.

Important : **Par exercice**, produire un fichier `.sql` comprenant vos requêtes SQL permettant de répondre aux différentes questions posées. Nommez ces fichiers `nom_prénom_exercice_1` et `nom_prénom_exercice_2`, (remplacez `nom` et `prénom` par votre nom et prénom).
A la fin du contrôle, envoyez vos deux fichiers `.sql` par e-mail à l'adresse l-raynal@ices.fr.
Chaque requête doit être exécutable sans modifications de ma part. Ajoutez une ligne de commentaire comme ci-dessous avant chaque requête afin d'indiquer la question concernée.

```
-- Question X
```

Répondre aux questions nécessitant une réponse manuscrite sur le sujet.

Exercice 1

L'objectif de cet exercice est de créer une base de données pour un cabinet médical. Vous devrez stocker des informations sur les consultations médicales ayant lieu dans ce cabinet, les médecins travaillant dans ce cabinet, et les patients de ce cabinet.

Il y a trois types d'entités qui vous intéressent et pour lesquels vous devrez stocker des informations : les médecins, les patients et les consultations. Vous devrez donc créer trois tables que l'on appellera `Médecins`, `Patients` et `Consultations`, afin de constituer votre base de données que vous nommerez `CabinetMédical`. Le cabinet souhaite recueillir certaines informations ci-dessous.

Pour un médecin, les informations :

- nom
- prénom
- date de naissance
- numéro de téléphone

- une information indiquant si le médecin travaille de manière permanente dans le cabinet ou non

Pour un patient, les informations :

- nom
- prénom
- ville de résidence
- code postal de résidence
- numéro de téléphone

Pour une consultation, les informations :

- identifiant du patient concerné par la consultation
- identifiant du médecin assurant la consultation
- date et heure de début de la consultation
- date et heure de fin de la consultation (lorsque connues)

Questions :

1. Complétez la liste des informations à recueillir afin que chaque table possède une clé primaire. Soulignez les informations qui auront le rôle de clés primaires.
2. Pour chaque information à stocker, indiquez le nom de colonne de votre choix que vous utiliserez dans les tables de la base de données.
3. Pour chaque information à stocker, écrire un type de colonne SQL approprié selon les données à stocker.
4. Pour chaque information à stocker, écrire les contraintes de colonnes SQL appropriées (s'il y en a) qui seront utilisées dans vos différentes tables.
5. Quelles sont les informations qui seront des clés étrangères dans les tables de votre base ?

6. Dessinez le schéma relationnel de votre base `CabinetMédical`. (Le dessiner sur le sujet, dans l'espace après la dernière question de cet exercice).
7. Créez une base de données vide nommée `CabinetMédical`, qui sera stockée dans un fichier `CabinetMédical.db` **sur le bureau de votre ordinateur**. Rédigez des requêtes de création de tables, afin de créer la structure des tables `Médecins`, `Patients` et `Consultations` en utilisant vos réponses aux questions précédentes.
8. Rédigez des requêtes d'insertion de données afin d'ajouter **deux** médecins fictifs dans la table `Médecins`. (Une requête par médecin est attendue).
9. Rédigez une unique requête d'insertion de données afin d'ajouter **deux** patients fictifs dans la table `Patients`.
10. Rédigez une requêtes d'insertion de données afin d'ajouter dans la table `Consultations` **une** consultation entre un patient et un médecin de votre base, prévue le 26 octobre 2024 à 15h30.
11. Rédigez une requête de mise à jour de données afin de modifier/préciser la date et l'heure de fin de la consultation créée en 10. pour le 26 octobre 2024 à 16h15.
12. Rédigez une requête de suppression de lignes afin de supprimer le médecin n'ayant pas donné de consultation.

Exercice 2

Cet exercice utilise une base de données nommée `DigitalMusic`, stockée dans le fichier `DigitalMusic.sqlite`, téléchargeable depuis le dossier `Examen1` à l'adresse https://github.com/LouisRaynal/coursSQL. Commencez par télécharger cette base de données **sur votre bureau**, et connectez-vous-y depuis SQLiteStudio.

Cette base de données contient des chansons disponibles sur une plateforme de vente en ligne. Deux tables constituent la base de données : `Chansons` et `Albums`.

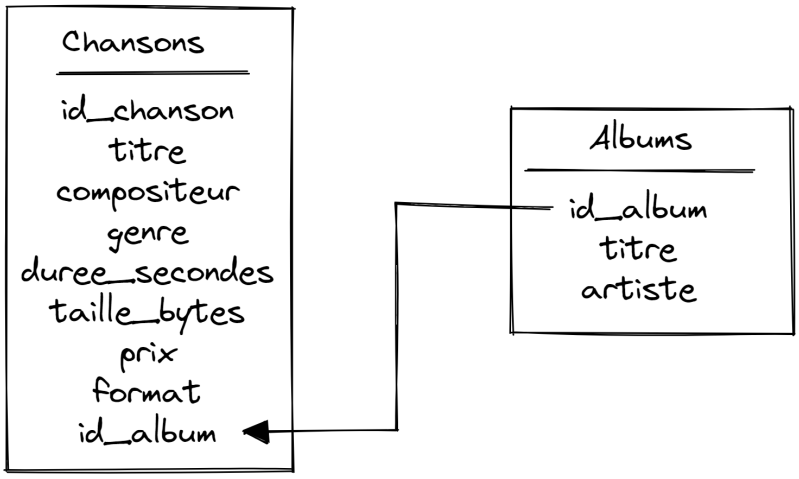
La table `Chansons` contient les informations suivantes pour chaque chanson :

- `id_chanson` - identifiant unique de la chanson ;
- `titre` - titre de la chanson ;
- `compositeur` - nom du ou des compositeurs ;
- `genre` - genre musical ;
- `duree_secondes` - durée en secondes ;
- `taille_bytes` - taille en bytes ;
- `prix` - prix de vente en euros ;
- `format` - format audio ;
- `id_album` - identifiant de l'album dans lequel figure la chanson.

La table `Albums` contient les informations suivantes pour chaque album :

- `id_album` - identifiant unique de l'album ;
- `titre` - titre de l'album ;
- `artiste` - artiste(s)/groupe.

Le schéma relationnel de cette base de données est représenté ci-dessous.



Questions :
Rédigez des requêtes SQL afin d'accomplir les tâches ci-dessous. Il vous faudra rédiger une requête par question.

1. Affichez toutes les informations des chansons pour lesquelles le compositeur n'est pas renseigné.
2. Affichez les formats audio différents utilisés pour les chansons de cette base de données.
3. Affichez les titres et genres des chansons dont le genre musical est soit `Pop`, soit `Blues` ou soit `Jazz`.
4. Affichez toutes les informations des chansons dont les compositeurs contiennent les caractères `Bowie` ou `Queen`.
5. Affichez les titres et durées en secondes des chansons dont la durée est comprise entre 60 et 120 secondes et dont le titre commence ou se termine par les caractères `me`.
6. Affichez les titres, genres, durées en secondes, tailles en bytes et le ratio (durée en secondes / taille en bytes) des chansons avec les 15 plus longues durées.
7. Affichez tous les titres de chansons en majuscules, ainsi que leurs durées **en minutes**. Ordonnez les lignes par durées décroissantes.
8. Affichez le titre de la chanson ayant le plus long titre dans le genre musical `Classical`, ainsi que la longueur de son titre. Nommez cette dernière colonne `longueur du titre`.
9. Pour chaque format audio, affichez combien de chansons sont disponibles à la vente, ainsi que la durée moyenne en secondes (en utilisant la fonction d'agrégation `AVG( )` pour calculer la moyenne).
10. Affichez tous les titres d'albums, artistes d'albums et titres de chansons correspondant. Dans votre requête vous définirez `al` et `ch` comme alias de table pour `Albums` et `Chansons` respectivement, et les emploierez lorsque possible.
11. Pour chaque titre d'albums, affichez combien de chansons sont contenues dans celui-ci. Triez les résultats par nombre de chansons décroissant.
12. La fonction `SUBSTR(c, n, m)` prend comme arguments une expression `c` s'évaluant comme une chaîne de caractères, puis deux entiers `n` et `m`. Cette fonction retourne `m` caractères de la chaîne, à partir de la position `n`.

    ```
    -- Par exemple
    SELECT SUBSTR('Bonjour', 1, 3)
    ;
    -- Retourne la chaîne de caractères 'Bon', car il s'agit des 3 caractères de la
    -- chaîne 'Bonjour' à partir de la position 1.
    ```

    Utilisez la fonction `SUBSTR` pour afficher les titres d'albums et les artistes ayant leurs 5 premiers caractères en commun.
13. L'opérateur `||` permet de concaténer deux chaînes de caractères entre lesquelles cet opérateur est placé.

    ```
    -- Par exemple
    SELECT 'Bonjour ' || 'le monde'
    ;
    -- Retourne la chaîne de caractères 'Bonjour le monde'
    ```

    En utilisant l'opérateur `||`, affichez tous les artistes, titres d'albums et une nouvelle colonne correspondant à la concaténation des artistes et titres d'albums en séparant ces deux informations par un `- `. (Par exemple : `AC/DC - Let There Be Rock`). Triez les résultats par ordre alphabétique croissant des artistes. Utilisez respectivement `Artiste`, `Album` et `Artiste - Album` comme alias de colonne.